Process Verification

Lab 2

Quentin Meneghini

13th March 2026

Facultad d'Informática de Barcelona

Contents

1 Design Overview **1**

1.1 Interface 1

1.2 Behavioral Rules 1

1.3 Functional Specifications 2

2 Objectives **3**

3 Traceability matrix **4**

4 Testbench Architecture **5**

4.1 Testbench Implementation 5

4.1.1 Key Components 5

4.1.2 Test Coverage 6

4.2 Automation and Scripting 6

5 Results **8**

6 Conclusion **9**

A Appendix Chapter **10**

A.1 Use of tools 10

A.2 Use of time 10

A.3 Use of AI 10

A.4 Testbench code 11

A.5 ADD wave forms 13

References **10**

Chapter 1

Design Overview

The Device Under Test (DUT) is a 4-bit Arithmetic Logic Unit (ALU) implemented as a purely combinational circuit. The ALU performs basic arithmetic and logic operations on two 4-bit operands A and B according to a 3-bit operation selector op.

1.1 Interface

The ALU interface consists of the following signals:

Signal	**Direction**	**Width**	**Description**
A	Input	4	Operand A
B	Input	4	Operand B
op	Input	3	Operation selector (Opcode)
Y	Output	4	ALU result
carry	Output	1	Carry/borrow out (valid for ADD/SUB)

Table 1.1: ALU singnals

1.2 Behavioral Rules

To ensure design correctness, the following specific rules must be observed:

1. **Carry Logic:** For the ADD operation, the carry bit acts as the 5th bit of the result and for the SUB the borrw flag if B is bigger than A.

2. **Carry Reset:** For all logical operations (AND, OR, XOR, NOT) and pass-through operations, the `carry` signal must be set to 0.

3. **Invalid Opcodes:** While all 3-bit combinations are defined in this spec, any invalid handling must result in $Y = 0000$ and $carry = 0$.

4. **Combinational Logic:** The design contains no clock or reset signals.

1.3 Functional Specifications

The ALU behavior is determined by the 3-bit *op* signal. The operations and their respective outputs are defined as follows:

Opcode (op)	**Operation**	**Description**	**Result (Y)**	**Carry bit (carry)**
000	ADD	Unsigned addition	$A + B$	MSB of 5-bit sum
001	SUB	Unsigned subtraction	$A - B$	Borrow flag
010	AND	Bitwise AND	$A \ \& \ B$	0
011	OR	Bitwise OR	$A \mid B$	0
100	XOR	Bitwise XOR	$A \oplus B$	0
101	NOT A	Bitwise negation of A	$\sim A$	0
110	PASS A	Output Operand A	A	0
111	PASS B	Output Operand B	B	0

Table 1.2: ALU Functional Descriptions

Chapter 2

Objectives

The primary goal of this laboratory is to perform the functional verification of 6 implementations of a 4-bit Arithmetic Logic Unit (ALU) using a black-box testing strategy (We do not have access to the DUT code). This objective can be divided into four pillars:

1. **Requirements:** define the requirements to be tested. It is important to include all the functional requirements and edge cases.
2. **Tests:** define the tests necessary to verify the requirements. It is important to define computationally easy tests that can cover the requirements.
3. **Implementation:** implement the tests to enable testing and apply them to the DUT.
4. **Report:** Create a report that the design office can use to fix any bugs found in this phase.

Chapter 3

Traceability matrix

For each requirement one or more tests are assigned to reach full coverage. the tests can be divided into two main categories:

- **Core logic:** correct behavior of output Y.
- **Carry logic:** correct behavior of output carry.

The requirements are further divided into each operation that the ALU is supposed to perform with the addition of the invalid input.

Req ID	Feature	Op	Requirement	Test ID
REQ_000	Core logic	ADD	Y correct for every input	T_000
REQ_001	Core logic	SUB	Y correct for every input	T_001
REQ_002	Core logic	AND	Y correct for every input	T_002
REQ_003	Core logic	OR	Y correct for every input	T_003
REQ_004	Core logic	XOR	Y correct for every input	T_004
REQ_005	Core logic	NOT A	Y correct for every input	T_005
REQ_006	Core logic	OUTPUT A	Y correct for every input	T_006
REQ_007	Core logic	OUTPUT B	Y correct for every input	T_007
REQ_008	Core logic	INVALID	Y set to 0000	T_008
REQ_009	Carry logic	ADD	carry correct for every input (overflow)	T_000
REQ_010	Carry logic	SUB	carry correct for every input (borrow)	T_001
REQ_011	Carry logic	AND	carry set to 0	T_002
REQ_012	Carry logic	OR	carry set to 0	T_003
REQ_013	Carry logic	XOR	carry set to 0	T_004
REQ_014	Carry logic	NOT A	carry set to 0	T_005
REQ_015	Carry logic	OUTPUT A	carry set to 0	T_006
REQ_016	Carry logic	OUTPUT A	carry set to 0	T_007
REQ_017	Carry logic	INVALID	Carry set to 0	T_008

Table 3.1: Traceability Matrix

Chapter 4

Testbench Architecture

4.1 Testbench Implementation

The implemented testbench, alu_4bit_tb.sv, utilizes a behavioral reference model to perform exhaustive verification of the ALU. The architecture is designed to be modular, allowing for easy switching between different DUT versions (*v0* through *v5*). the full code is available in the appendix [**<empty citation>**].

4.1.1 Key Components

- **Reference Model (assert_golden_model):** This task acts as the "Golden Model." It calculates the expected Y and *carry* values based on the design specification for every operation and tests it against the obtained values.

Listing 4.1: assert_golden_model code

```
task assert_golden_model(
    input op_e  operand,
    input int   a,
    input int   b
);
    logic [I_SIZE -1 : 0]   expected_Y;
    logic                   expected_carry;

    case (operand)
        ADD:    begin expected_Y = a + b; expected_carry = (a + b) >> I_SIZE; end
        SUB:    begin expected_Y = a - b; expected_carry = (a - b) >> I_SIZE; end
        AND:    begin expected_Y = a & b; expected_carry = 0;                 end
```

```
        OR:       begin expected_Y = a | b; expected_carry = 0;                end
        XOR:      begin expected_Y = a ^ b; expected_carry = 0;                end
        NOT_A:    begin expected_Y = ~a;    expected_carry = 0;                end
        PASS_A:   begin expected_Y = a;     expected_carry = 0;                end
        PASS_B:   begin expected_Y = b;     expected_carry = 0;                end
        default:  begin expected_Y = 0;     expected_carry = 0;                end
    endcase

    assert (Y == expected_Y && carry == expected_carry)
        else $error("%s failed for A=%0d, B=%0d: Y=%0d, carry=%b", operand, A, B, Y, carry);
endtask
```

- **Exhaustive Stimulus (test_op):** Because the design space is small ($2^4 \times 2^4 = 256$ combinations), the testbench uses nested loops to test every possible input combination for every operation.

Listing 4.2: test_op code

```
task test_op(
    input op_e operand
);
    op = operand;
    for (int a = 0; a < (1 << I_SIZE); a++) begin
        for (int b = 0; b < (1 << I_SIZE); b++) begin
            A = a;
            B = b;
            #1;
            assert_golden_model(operand, a, b);
        end
    end
endtask
```

4.1.2 Test Coverage

The testbench executes nine distinct test suites (`T_000` to `T_008`). This includes all eight functional opcodes and a specific test for undefined/invalid input handling. Thus reaching the 100% coverage of the module.

4.2 Automation and Scripting

To streamline the verification of the six different RTL implementations provided (*v0* to *v5*), a modular TCL script (run.do) was developed.

Listing 4.3: run.do code

```
# version
set VERSION $1
set FULL_NAME "alu_4bit_v$VERSION"

# compilation
vlog +acc +define+DUT_NAME=$FULL_NAME alu_4bit_tb.sv
vsim work.alu_4bit_tb

# execution
add wave *
restart -f
run -all
```

Chapter 5

Results

After running the tests on each implementation (v0 - v5) we found out that half of the implementations were correct, while the second half was wrong in some way. The failed requirements are listed in Table 5.1. In particular after analyzing these results:

- **v2:** failed because SUB operations produce wrong results both in terms of Y and *carry*.
- **v3:** failed beacuse AND and OR operations are inverted, leading to 120 operation wrong each (got right answere whenever the two make the same results).
- **v5:** failed because ADD operation add 1 to each result Y. Furthermore XOR operation produces errors on the output, but correctely keeps *carry* to 0

Version	**num err**	**Failed Requirements**
`alu_4bit_v0`	0	**PASS**: Correct RTL design implementation.
`alu_4bit_v1`	0	**PASS**: Correct RTL design implementation.
`alu_4bit_v2`	120	**FAIL**: REQ_001 — REQ_010.
`alu_4bit_v3`	480	**FAIL**: REQ_002 — REQ_003.
`alu_4bit_v4`	0	**PASS**: Correct RTL design implementation.
`alu_4bit_v5`	431	**FAIL**: REQ_000 — REQ_004 — REQ_009.

Table 5.1: Verification Results

Chapter 6

Conclusion

This laboratory session successfully performed the functional verification of six 4-bit ALU implementations using a black-box testing strategy. By developing an exhaustive, self-checking SystemVerilog testbench, 100% functional coverage was achieved across all eight operations and invalid opcode handling.

The verification process confirmed that versions v0, v1, and v4 are fully compliant with the design specifications. Critical functional bugs were identified in the remaining designs:

1. **v2:** Failed due to incorrect subtraction results (Y) and carry logic (REQ_001, REQ_010).
2. **v3:** Failed because the AND and OR operations were swapped (REQ_002, REQ_003).
3. **v5:** Failed due to an arithmetic offset in addition and the implementation of XOR is wrong (REQ_000, REQ_004, REQ_009).

The use of a parameterized TCL script for automation and a modular testbench architecture proved highly effective, allowing for a streamlined verification flow and rapid identification of RTL discrepancies.

Appendix A

Appendix Chapter

A.1 Use of tools

- Visual Studio Code: systemverilog IDE.
- QuestaSim: tester.
- GTKwave: wave visualizer.

A.2 Use of time

For this lab, approximately 13 hours were used.

- Understanding the request: 1,5 h
- resolving the lab: 1h
- writing the report: 3h
- getting AulaVirtual to work: 6h
- questasim installation (failed due to update of the platform): 1,5h

A.3 Use of AI

- Gemini (Pro): Sentence refining, debugging AulaVirtual and questasim installation problems, snippets automation.

A.4 Testbench code

Listing A.1: TestBench code

```
`timescale 1ns/1ps

`ifndef DUT_NAME
    `define DUT_NAME alu_4bit_v0
`endif

module alu_4bit_tb;

    localparam int I_SIZE = 4;
    localparam int O_SIZE = 3;

    logic [I_SIZE -1 : 0]   A, B, Y;
    logic [O_SIZE -1 : 0]   op;
    logic                   carry;

    `DUT_NAME dut (
        .A(A),
        .B(B),
        .op(op),
        .Y(Y),
        .carry(carry)
    );

    typedef enum logic [O_SIZE -1 : 0] {
        ADD =       3'b000,
        SUB =       3'b001,
        AND =       3'b010,
        OR  =       3'b011,
        XOR =       3'b100,
        NOT_A =     3'b101,
        PASS_A =    3'b110,
        PASS_B =    3'b111
    } op_e;

    task assert_golden_model(
        input op_e  operand,
        input int   a,
        input int   b
    );
        logic [I_SIZE -1 : 0] expected_Y;
        logic expected_carry;
        string op_str;
```

```
        case (operand)
            ADD:     begin  expected_Y = a + b;     expected_carry = (a + b) >> I_SIZE;
op_str = "ADD";     end
            SUB:     begin  expected_Y = a - b;     expected_carry = (a - b) >> I_SIZE;
op_str = "SUB";     end
            AND:     begin  expected_Y = a & b;     expected_carry = 0;
op_str = "AND";     end
            OR:      begin  expected_Y = a | b;     expected_carry = 0;
op_str = "OR";      end
            XOR:     begin  expected_Y = a ^ b;     expected_carry = 0;
op_str = "XOR";     end
            NOT_A:   begin  expected_Y = ~a;        expected_carry = 0;
op_str = "NOT_A";   end
            PASS_A:  begin  expected_Y = a;         expected_carry = 0;
op_str = "PASS_A";  end
            PASS_B:  begin  expected_Y = b;         expected_carry = 0;
op_str = "PASS_B";  end
            default: begin  expected_Y = 0;         expected_carry = 0;
op_str = "INVALID"; end // invalid
        endcase

        assert (Y == expected_Y && carry == expected_carry)
            else $error("%s failed for A=%0d, B=%0d: Y=%0d, carry=%b", op_str, A, B, Y, carry);
    endtask

    task test_op(
        input op_e operand
    );
        op = operand;
        for (int a = 0; a < (1 << I_SIZE); a++) begin
            for (int b = 0; b < (1 << I_SIZE); b++) begin
                A = a;
                B = b;
                #1;
                assert_golden_model(operand, a, b);
            end
        end
    endtask

    initial begin
        #10; test_op(ADD);          // T_000
        #10; test_op(SUB);          // T_001
        #10; test_op(AND);          // T_002
        #10; test_op(OR);           // T_003
```

```
        #10; test_op(XOR);          // T_004
        #10; test_op(NOT_A);        // T_005
        #10; test_op(PASS_A);       // T_006
        #10; test_op(PASS_B);       // T_007
        #10; test_op(op_e'('bxxx)); // T_008
    end

    initial begin
        $dumpfile("waves%s.vcd", `DUT_NAME);
        $dumpvars(0, alu_4bit_tb);
    end

endmodule
```

A.5 ADD wave forms

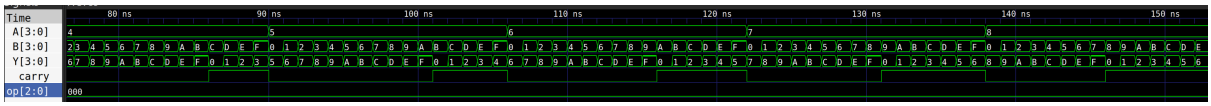

Figure A.1: v0

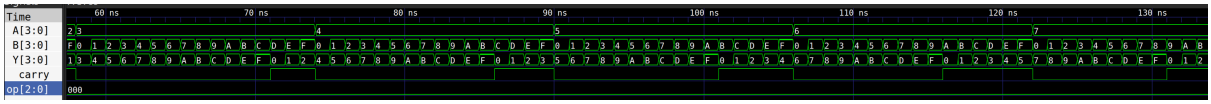

Figure A.2: v1

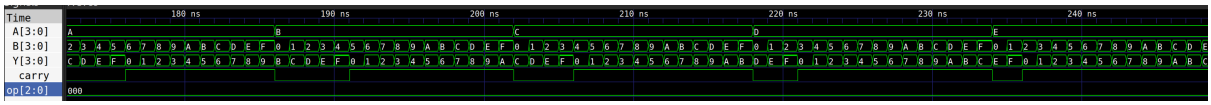

Figure A.3: v2

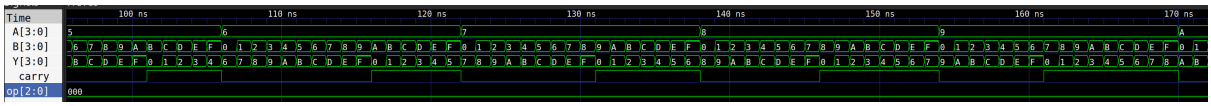

Figure A.4: v3

Figure A.5: v4

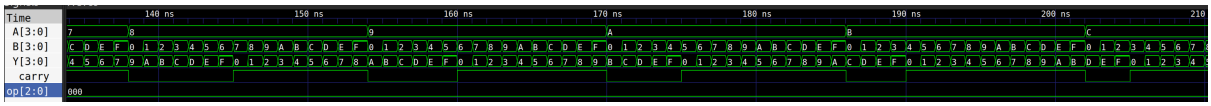
Time
A[3:0]
B[3:0]
Y[3:0]
carry
op[2:0]
140 ns
150 ns
160 ns
170 ns
180 ns
190 ns
200 ns
210
000

Figure A.6: v5